

Memory allocation and management in CUDA C/C++ applications.

GPU programming in computational engineering

Escuela de Doctorado 2025/26

University of Zaragoza

Sergio Martínez-Aranda

sermar@unizar.es

Mario Morales-Hernández

mmorales@unizar.es

Fluid Dynamics Technologies - I3A, University of Zaragoza



Universidad
Zaragoza



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Instituto Universitario de Investigación
en Ingeniería de Aragón
Universidad Zaragoza

Module content

1. Remembering basic concepts
2. First steps in CUDA programming
3. Memory management optimization

Module content

1. Remembering basic concepts
2. First steps in CUDA programming
3. Memory management optimization

GPU architecture

Thread

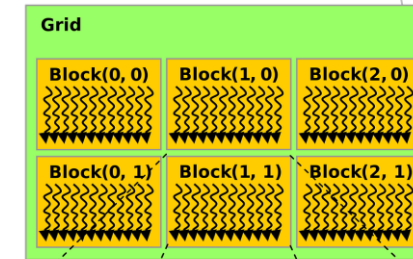
- The thread is the smallest unit of execution.
- Thousands of threads can run simultaneously on the GPU kernels.

Block

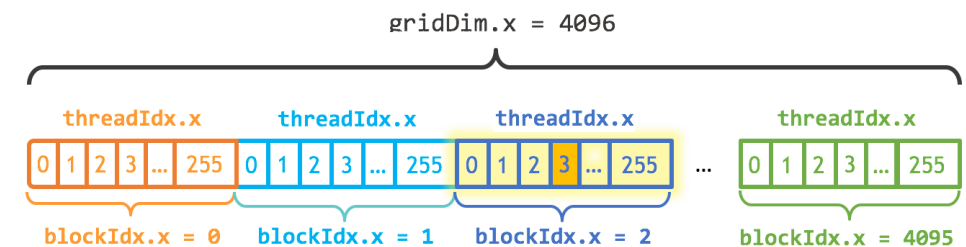
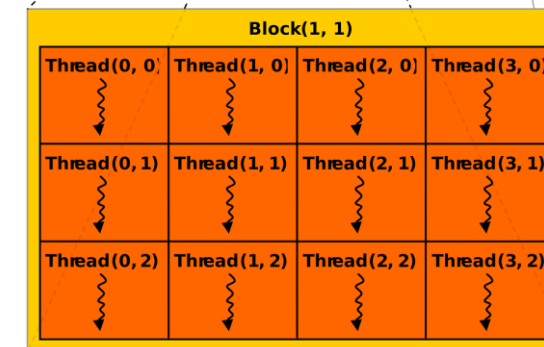
- Threads are grouped into blocks.
- Blocks are scheduled in the Streaming Multiprocessors (SM).
- Threads in the block are executed in warps (sets of 32 threads).

Grid

- The computation domain is defined by the grid, i.e. collection of blocks that execute the same kernel function.
- Each block can establish its identifier within the grid.
- Each thread can establish its unique identifier within the block.



(Reproduced from
<https://maori.geek.nz, 2021>)



$$\text{index} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

$$\text{index} = (2) * (256) + (3) = 515$$

GPU memory hierarchy

RAM memory (CPU)

- Large memory shared across all the CPU cores.
- Optimized for random access patterns and complex control flows.
- Primary working memory for I/O communications.

Global Memory

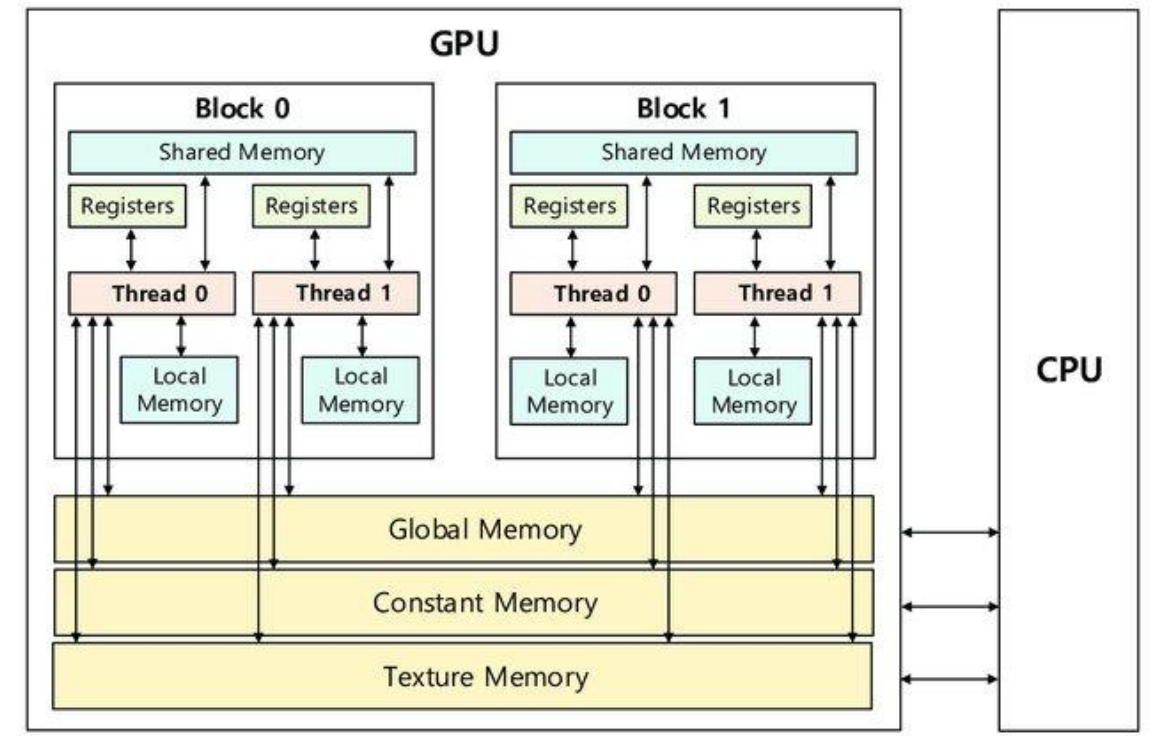
- High-capacity memory but with higher latency.
- Accessible by all the threads in the grid.

Shared Memory

- Faster, on-chip memory shared among the threads of each blocks.
- Limited to 48KB per block in most of GPUs.

Registers

- Ultra-fast memory dedicated to individual threads.
- Limited to 255 registers per thread.

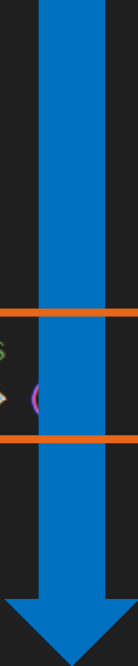


(Reproduced from An and Seo, 2020)

CUDA/C++ code workflow

```
23
24 // Driven function executed in CPU
25 int main()
26 {
27     ...
28     RAM memory allocation and initialization
29     ...
30     GPU selection and setup
31     ...
32     CUDA global memory allocation
33     ...
34     RAM --> CUDA data transfer
35     ...
36     // Kernel invocation with multi-threads
37     say_hi <<<numBlocks, threadsPerBlock>>> (
38     ...
39     CUDA --> RAM memory transfer
40     ...
41     Write output files
42     ...
43 }
44
```

Sequential workflow



CUDA kernel parallelized in the GPU device.

CUDA/C++ code workflow

Sequential
workflow



```
23
24 // Driven function executed in CPU
25 int main()
26 {
27     ...
28     RAM memory allocation and initialization
29     ...
30     GPU selection and setup
31     ...
32     CUDA global memory allocation
33     ...
34     RAM --> CUDA data transfer
35     ...
36     // Kernel invocation with multi-threads
37     say_hi <<<numBlocks, threadsPerBlock>>> (
38     ...
39     CUDA --> RAM memory transfer
40     ...
41     Write output files
42     ...
43 }
44
```

CUDA memory allocation and initialization.

CUDA kernel parallelized in the GPU device.

CUDA/RAM memory transfer.

CUDA/C++ compilation and execution

- CUDA/C++ code is written in `app_code.cu` files, including the header:

```
#include <cuda_runtime.h>
```

- NVCC compiler is used to create executable files:

```
nvcc -arch=sm_XX -O3 -G -o exe_app app_code.cu -lXXXXX
```

- Additional flags for `nvcc` compiler:

`-arch=sm_XX` to specify the compute capability of the GPU device

`-G` to include memory check information for debugging tools.

`-O3` to enable high-level optimizations

`-lXXXXX` to include external libraries, such as `-lcudart` or `-lcublas`.

- The compiled application runs with the command:

```
./exe_app
```

- `nvidia-smi` command provides performance data of the GPU during the execution of the application.

Working in the HERMES2 supercomputing cluster

HERMES supercomputing cluster is the scientific infrastructure dedicated for researchers at the Institute of Engineering Research (I3A), University of Zaragoza. HERMES has over 3,260 CPU cores, with 5,920 parallel processing threads and 26 TB of RAM. HERMES features 251,392 CUDA cores (including NVIDIA A100 GPUs) and 720 GB of dedicated memory.



Instituto Universitario de Investigación
en Ingeniería de Aragón
Universidad Zaragoza

CUDA hands-on sessions

Dedicated computation node with 4 x NVIDIA A100 devices.
20 working users. Each participant has its own user assigned

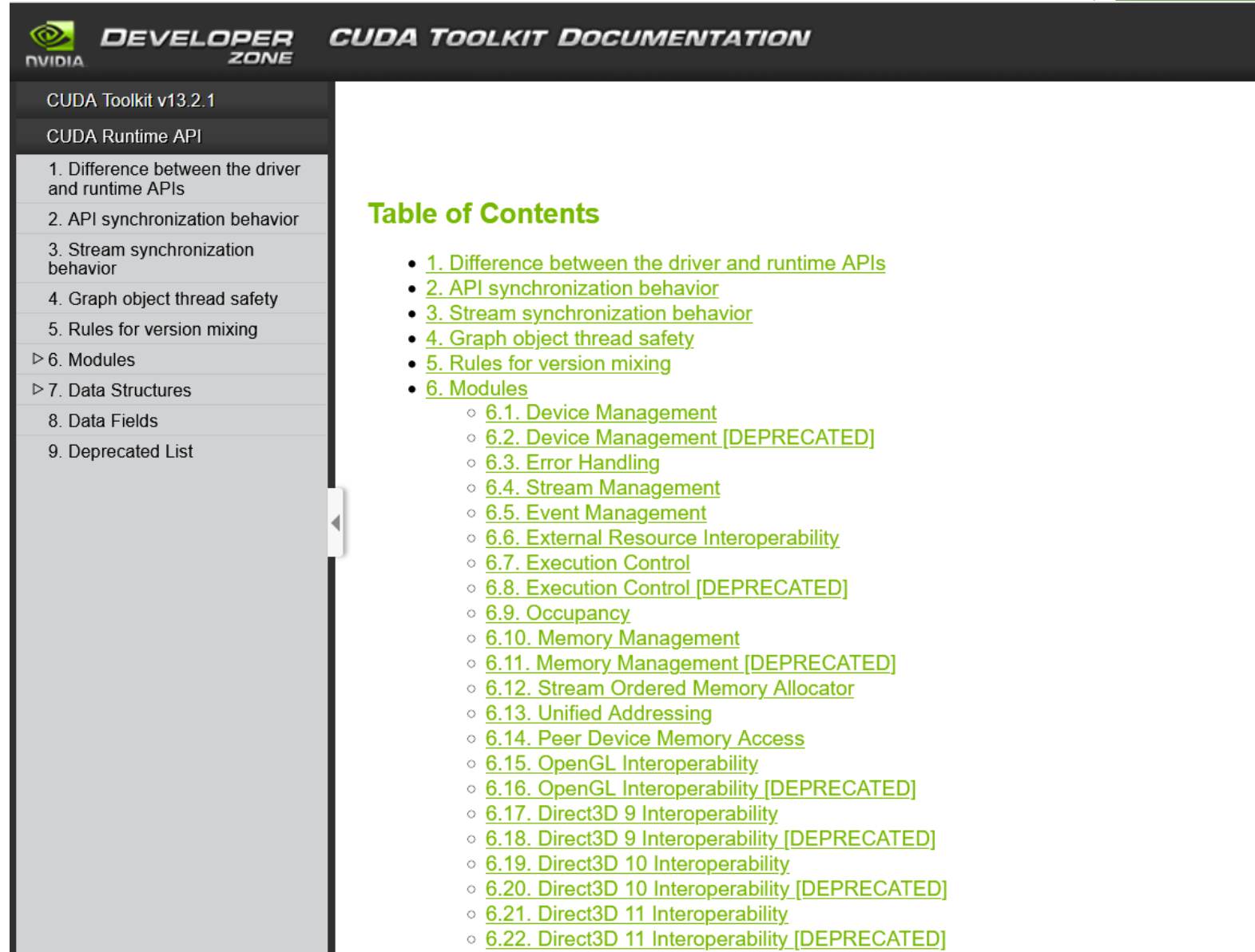
- Remote access by Ubuntu terminal:
`ssh workertfd<X>@155.210.134.18 -p 4001`
- User: `workertfd<X>` with `X=2,...,20`
Note: `workertfd1` is reserved for professors.
- Password: `Workertfd2026`

Module content

1. Remembering basic concepts
2. First steps in CUDA programming
3. Memory management optimization

CUDA Toolkit

Runtime API



The screenshot shows the NVIDIA Developer Zone CUDA Toolkit Documentation page for the Runtime API. The page has a dark header with the NVIDIA logo, 'DEVELOPER ZONE', and 'CUDA TOOLKIT DOCUMENTATION'. The left sidebar contains a navigation menu for 'CUDA Toolkit v13.2.1' and 'CUDA Runtime API', listing topics from 1 to 9. The main content area is titled 'Table of Contents' and lists the same topics as a bulleted list of links. Topics 6 through 22 are indented under topic 6, 'Modules'.

CUDA Toolkit v13.2.1
CUDA Runtime API

- 1. Difference between the driver and runtime APIs
- 2. API synchronization behavior
- 3. Stream synchronization behavior
- 4. Graph object thread safety
- 5. Rules for version mixing
- ▷ 6. Modules
- ▷ 7. Data Structures
- 8. Data Fields
- 9. Deprecated List

Table of Contents

- [1. Difference between the driver and runtime APIs](#)
- [2. API synchronization behavior](#)
- [3. Stream synchronization behavior](#)
- [4. Graph object thread safety](#)
- [5. Rules for version mixing](#)
- [6. Modules](#)
 - [6.1. Device Management](#)
 - [6.2. Device Management \[DEPRECATED\]](#)
 - [6.3. Error Handling](#)
 - [6.4. Stream Management](#)
 - [6.5. Event Management](#)
 - [6.6. External Resource Interoperability](#)
 - [6.7. Execution Control](#)
 - [6.8. Execution Control \[DEPRECATED\]](#)
 - [6.9. Occupancy](#)
 - [6.10. Memory Management](#)
 - [6.11. Memory Management \[DEPRECATED\]](#)
 - [6.12. Stream Ordered Memory Allocator](#)
 - [6.13. Unified Addressing](#)
 - [6.14. Peer Device Memory Access](#)
 - [6.15. OpenGL Interoperability](#)
 - [6.16. OpenGL Interoperability \[DEPRECATED\]](#)
 - [6.17. Direct3D 9 Interoperability](#)
 - [6.18. Direct3D 9 Interoperability \[DEPRECATED\]](#)
 - [6.19. Direct3D 10 Interoperability](#)
 - [6.20. Direct3D 10 Interoperability \[DEPRECATED\]](#)
 - [6.21. Direct3D 11 Interoperability](#)
 - [6.22. Direct3D 11 Interoperability \[DEPRECATED\]](#)

Main step in CUDA/C++ code workflow

**Sequential
workflow**



GPU selection and setup

CUDA memory allocation

CPU-GPU data transfer

CUDA kernel execution

GPU selection and setup

```
48 |  
49 int numGPUs = 0;  
50 cudaGetDeviceCount(&numGPUs);  
51
```

Get the number of available GPUs with computing capabilities.

```
51  
52 int gpu_id = 0;  
53 cudaDeviceProp props;  
54 cudaGetDeviceProperties(&props, gpu_id);  
55
```

Get the specific properties of the selected GPU device. The input `gpu_id` is type integer. The output is class-type `cudaDeviceProp`.

GPU selection and setup

```
48  
49 int numGPUs = 0;  
50 cudaGetDeviceCount(&numGPUs);  
51
```

Get the number of available GPUs with computing capabilities.

```
51  
52 int gpu_id = 0;  
53 cudaDeviceProp props;  
54 cudaGetDeviceProperties(&props, gpu_id);  
55
```

Get the specific properties of the selected GPU device. The input `gpu_id` is type integer. The output is class-type `cudaDeviceProp`.

```
55  
56 int selectedGPU = 0;  
57 cudaSetDevice(selectedGPU);  
58
```

Select a specific GPU device to run the parallel CUDA kernels.

```
58  
59 size_t freeMemory, totalMemory;  
60 cudaMemGetInfo(&freeMemory, &totalMemory);  
61
```

Get the total available memory and the free memory for a specific GPU device.

```
61  
62 cudaDeviceSynchronize();  
63
```

It ensures all the CUDA tasks on the GPU device have been completed before the host (CPU) continues execution.

Main step in CUDA/C++ code workflow

**Sequential
workflow**



GPU selection and setup

CUDA memory allocation

CPU-GPU data transfer

CUDA kernel execution

CUDA memory allocation: Global memory

```
66
67 int main() {
68
69     // Host memory
70     size_t size = N*sizeof(double);
71     double *h_A;
72     h_A = (double*) malloc(size);
73
74     ...
75     double *d_A;
76     cudaMalloc((void**) &d_A, size);
77
78
79     int threadsPerBlock = 256;
80     int blocksPerGrid = N/threadsPerBlock+1;
81
82     memoryCheck
83     <<<blocksPerGrid, threadsPerBlock>>>
84     (N, size, d_A);
85
86
87     cudaFree(d_A);
88     ...
89 }
90
```

Dynamic allocation of computation arrays in the host (RAM) memory

Dynamic allocation of computation arrays in the GPU global memory

Parallelized kernel executed in the CUDA grid

Free GPU global memory

CUDA memory allocation: Thread register memory

```
89
90 __global__ void registerMemoryAllocate(int N, size_t size, double *d_A) {
91     double rA;
92
93     int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
94     if (idx < N) {
95         rA = (double)(idx);
96         d_A[idx] = rA;
97     }
98 }
99
100
101 int main() {
102     ...
103     int threadsPerBlock = 256;
104     int blocksPerGrid = N/threadsPerBlock+1;
105
106     registerMemoryAllocate
107         <<<blocksPerGrid, threadsPerBlock>>>
108         (N, size, d_A);
109     ...
110 }
111
```

Static register memory allocation

Parallel computation of the register at each thread and output to the global memory array.

Parallelized kernel executed in the CUDA grid

CUDA memory allocation: Block shared memory

```
118
119 __global__ void sharedMemoryAllocate(int N, size_t size, double *d_A) {
120
121     int ithread = threadIdx.x;
122     int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
123
124     extern __shared__ double shared_A[];
125     if (idx < N) {
126         shared_A[ithread] = (double)(idx);
127     }
128     __syncthreads();
129
130     if (idx < N) {
131         d_A[idx] = shared_A[ithread];
132     }
133 }
134
135 int main() {
136     ...
137     int threadsPerBlock = 256;
138     int blocksPerGrid = N/threadsPerBlock+1;
139     size_t requiredSharedMemory = threadsPerBlock*sizeof(double);
140
141     sharedMemoryAllocate
142         <<<blocksPerGrid, threadsPerBlock, requiredSharedMemory>>>
143         (N, size, d_A);
144     ...
145 }
146
```

Dynamical allocation of the shared memory. The idx thread computes to the ithread position in the shared array of the blockIdx.x block. __syncthreads() ensures that all threads have finished writing the shared array.

Parallel output to the global memory array.

Parallelized kernel executed in the CUDA grid. The size of the shared memory should be provided in the CUDA grid definition.

Main step in CUDA/C++ code workflow

**Sequential
workflow**



GPU selection and setup

CUDA memory allocation

CPU-GPU data transfer

CUDA kernel execution

CPU-GPU data transfer

```
153
154 int main() {
155
156     size_t size = N*sizeof(double); // arrays size
157
158     double *h_A = (double*) malloc(size); // host array
159
160     double *d_A;
161     cudaMalloc((void**) &d_A, size); // device array
162
163     ...
164     cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
165
166
167     vectorAdd <<<blocksPerGrid, threadsPerBlock>>>> (N, d_A, d_B);
168
169     cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceToHost);
170
171     ...
172 }
173
```

Copy array from host to global memory in the device

Copy array from device global memory back to host memory

CPU-GPU data transfer

```
153
154 int main() {
155     size_t size = N*sizeof(double); // arrays size
156     double *h_A = (double*) malloc(size); // host array
157
158     double *d_A;
159     cudaMalloc((void**) &d_A, size); // device array
160
161     ...
162     cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
163
164     vectorAdd <<<blocksPerGrid, threadsPerBlock>>> (N, d_A, d_B);
165
166     ...
167     cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceToHost);
168     ...
169 }
170
171
172
173
```

Copy array from host to global memory in the device

Copy array from device global memory back to host memory

```
149
150 cudaMemcpy(d_B, 0, size);
151 cudaMemcpy(d_D, 0xFF, N*sizeof(int)); //integer array
152
```

Set global memory to specific values (byte level).

Main step in CUDA/C++ code workflow

**Sequential
workflow**



GPU selection and setup

CUDA memory allocation

CPU-GPU data transfer

CUDA kernel execution

CUDA kernel execution: Grid setup

```
177
178 __global__ void arrayComputation(int N, double *A) {
179
180     int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
181     if (idx < N) {
182         A[idx] = ...
183     }
184 }
185
186 int main() {
187
188     int N = 1000;
189
190     double *d_A;
191     cudaMalloc((void**) &d_A, N*sizeof(double)); // device array
192
193     ...
194     arrayComputation <<<1, N>>> (N, d_A);
195
196
197
198     int threadsPerBlock = 256;
199     int blocksPerGrid = N/threadsPerBlock + 1;
200
201     arrayComputation <<<blocksPerGrid, threadsPerBlock>>> (N, d_A);
202     ...
203 }
204
```

__global__ function executed in the CUDA grid.

Number of tasks to execute in the parallel kernel, i.e. number of threads in the CUDA grid.

Single block execution.

CUDA kernel execution: Grid setup

```
177
178 __global__ void arrayComputation(int N, double *A) {
179
180     int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
181     if (idx < N) {
182         A[idx] = ...
183     }
184 }
185
186 int main() {
187
188     int N = 1000;
189
190     double *d_A;
191     cudaMalloc((void**) &d_A, N*sizeof(double)); // device array
192
193     ...
194     arrayComputation <<<1, N>>> (N, d_A);
195
196
197
198     int threadsPerBlock = 256;
199     int blocksPerGrid = N/threadsPerBlock + 1;
200
201     arrayComputation <<<blocksPerGrid, threadsPerBlock>>> (N, d_A);
202     ...
203 }
204
```

__global__ function executed in the CUDA grid.

Number of tasks to execute in the parallel kernel, i.e. number of threads in the CUDA grid.

Multiple block execution.

CUDA kernel execution: Device functions

```
238
239 __device__ void addDoubleData(double *A, double *B, double *C) {
240     (*C) = (*A) + (*B);
241 }
242
243 __global__ void deviceVectorAdd(int N, double *A, double *B, double *C) {
244
245     int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
246     if (idx < N) {
247         addDoubleData( &(A[idx]), &(B[idx]), &(C[idx]) );
248     }
249 }
250
251 int main() {
252
253     int N = 1000;
254
255     double *d_A;
256     cudaMalloc((void**) &d_A, N*sizeof(double)); // device array
257
258     ...
259     int threadsPerBlock = 256;
260     int blocksPerGrid = N/threadsPerBlock + 1;
261
262     deviceVectorAdd <<<blocksPerGrid, threadsPerBlock>>> (N, d_A, d_B);
263     ...
264 }
265
```

`__device__` function for algebraic in-thread operations.

`addDoubleData` is executed in each thread.

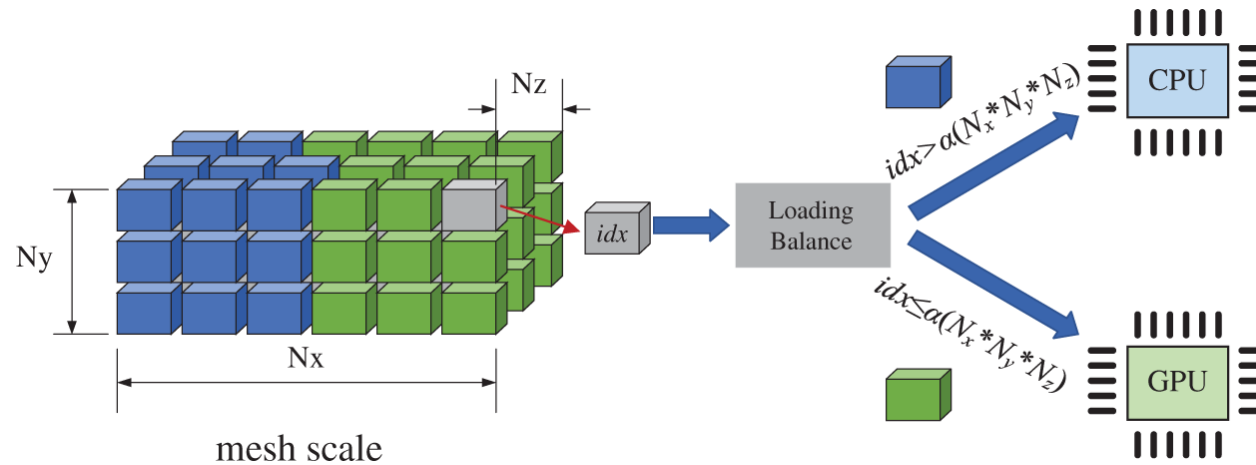
Multiple blocks parallel kernel

Module content

1. Remembering basic concepts
2. First steps in CUDA programming
3. Memory management optimization

Performance optimization strategies

- Vectorization
- Memory access optimization
- Kernel/subroutine implementation
- Thread management and load balancing
- Computation optimization and best practices



Vectorization

Vectorization refers to the process of organizing computations to take advantage of the SIMD (Single Instruction, Multiple Data) nature of GPU hardware.

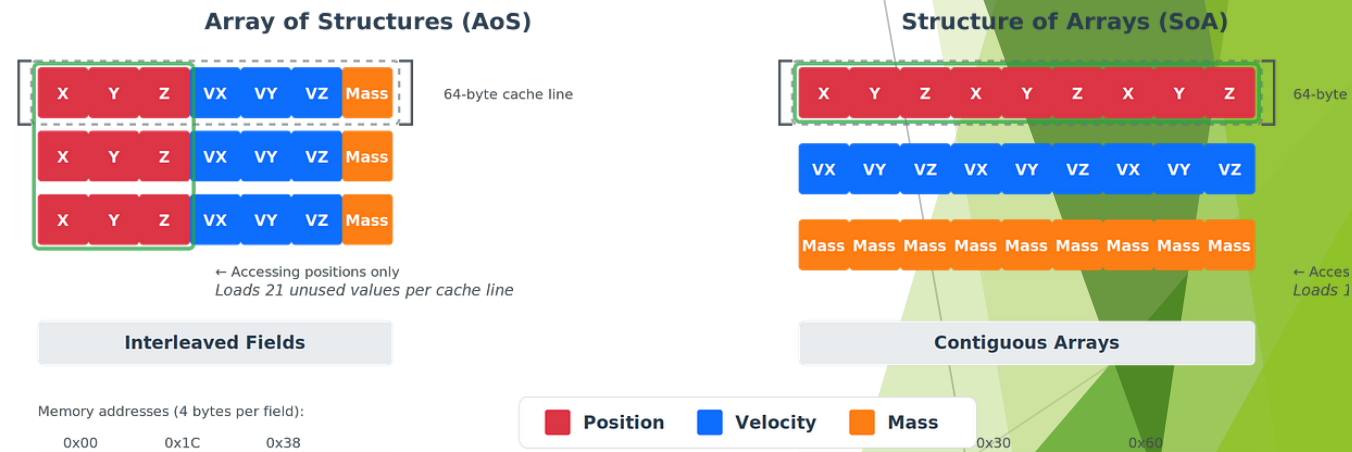
Data Organization

- Arrays-of-Structures (AoS) vs Structures-of-Arrays (SoA)
- Align data for coalesced memory access
- Pack data for SIMD operations

Computation patterns

- Loop unrolling for vector operations
- Vectorization-Friendly Patterns
- Replace sequential dependencies with parallel alternatives
- Use tiling and blocking for better data locality
- Minimize divergent execution paths

Array of Structures vs Structure of Arrays



Performance Impact

Array of Structures

- Poor cache utilization
- Wastes bandwidth on unused fields
- Limited SIMD efficiency
- + Good for accessing all fields
- + Simple memory layout

Structure of Arrays

- + Excellent cache utilization
- + Optimal bandwidth usage
- + Perfect for SIMD operations
- + Up to 40x faster for GPU
- ! More complex indexing

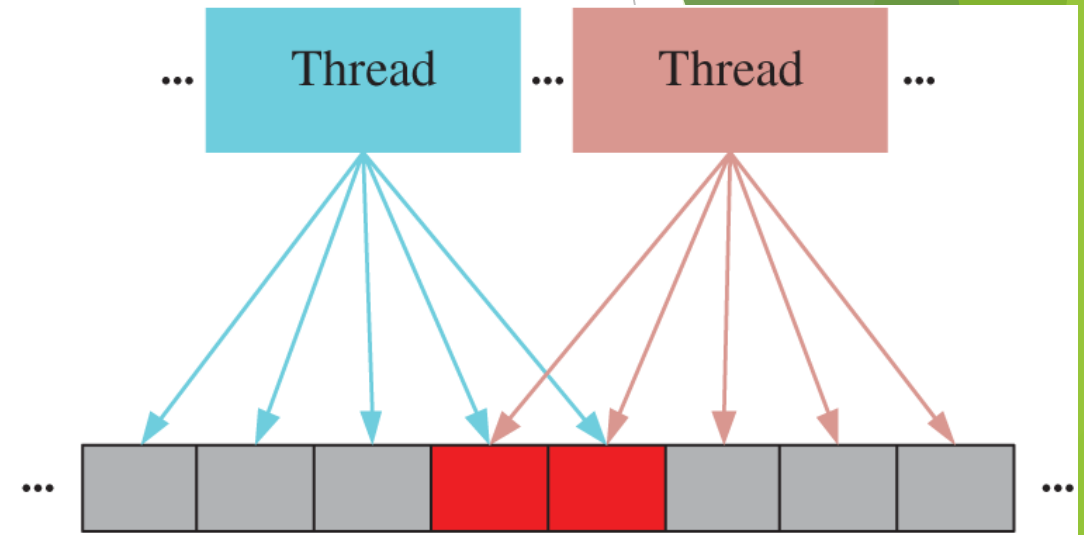
Vectorization & Race conditions

A 'race condition' arises if two or more threads access the same variables or objects concurrently and at least one does updates.

Example:

Suppose t_1 and t_2 simultaneously execute the statement $x = x + 1$; for some static global x .

- Internally, this involves loading x , adding 1, and storing x .
- If t_1 and t_2 do this concurrently, the statement is executed twice, but x may only be incremented once.
- t_1 and t_2 'race' to do the update.



Vectorization & Race conditions

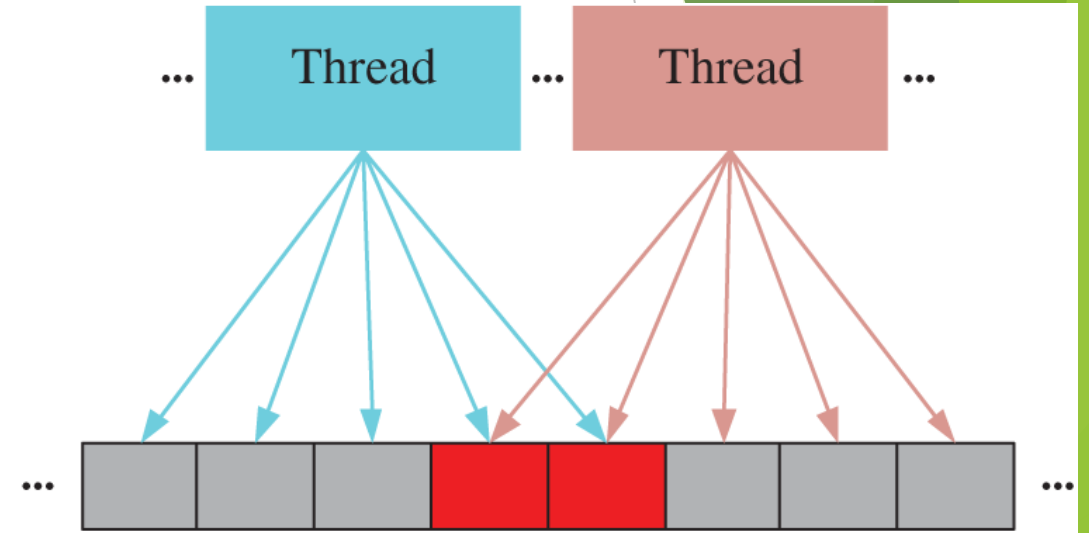
A 'race condition' arises if two or more threads access the same variables or objects concurrently and at least one does updates.

Common Race Conditions

- Multiple threads updating shared memory
- Concurrent access to boundary elements
- Reduction operations (sum, max, min)

Prevention Strategies

- Atomic operations for critical sections
- Memory barriers and synchronization
- Thread-local storage for intermediate results



Coalesced memory access

Coalesced access means that adjacent threads should access adjacent memory locations.

Example:

Thread 0 in Block 0 (warp 0) → Address N,

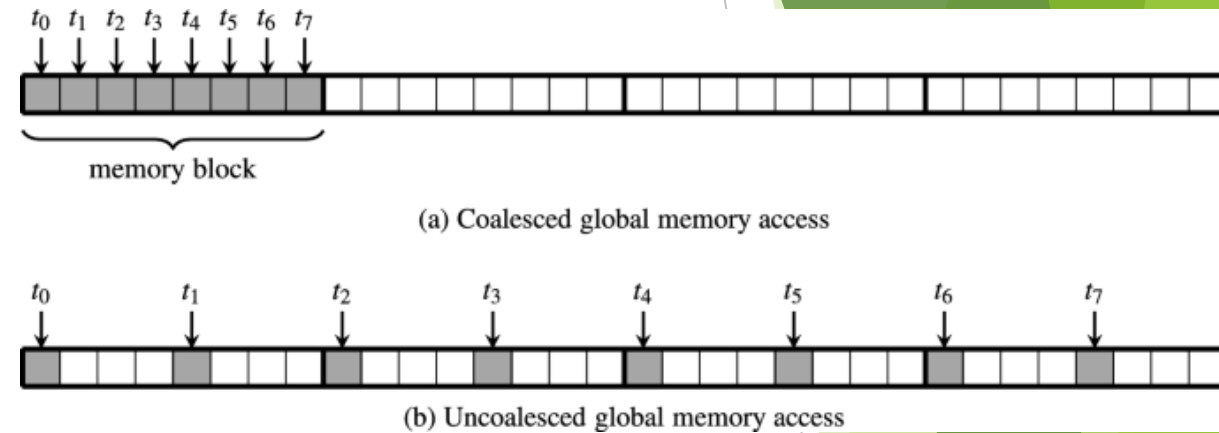
Thread 1 in Block 0 (warp 0) → Address N+1

...

Thread 31 in Block 0 (warp 0) → Address N+31

Advantages

- Higher memory bandwidth efficiency: Contiguous thread accesses are combined into fewer memory transactions.
- Lower memory access latency: Coalesced accesses reduce the number of separate global memory operations.
- Better kernel performance: GPUs spend less time waiting for memory and more time computing.



GPU kernel execution: Coalesced memory access

```
209
210 __global__ void nonCoalescentVectorAdd(int N, double *A, double *B) {
211
212     int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
213     if (idx < N) {
214         A[idx] = B[idx*stride];
215     }
216 }
217
218 int main() {
219
220     int N = 1000;
221     int stride = 8;
222
223     double *d_A;
224     cudaMalloc((void**) &d_A, N*sizeof(double)); // device array
225
226     double *d_B;
227     cudaMalloc((void**) &d_B, stride*N*sizeof(double)); // device array
228
229     ...
230     int threadsPerBlock = 256;
231     int blocksPerGrid = N/threadsPerBlock + 1;
232
233     nonCoalescentVectorAdd <<<blocksPerGrid, threadsPerBlock>>> (N, d_A, d_B);
234     ...
235 }
236
```

Coalesced memory access to A array
Non-coalesced memory access to B array.

Coalesced array

Non-coalesced array

Multiple blocks parallel kernel

Thread management

Thread Block Optimization

- Optimal block size: Multiple of 32 (warp size).
- Common sizes: 128, 256, or 512 threads per block.
- Balance block size with resource usage.

Common Pitfalls

- Thread divergence in loops with varying iterations.
- Unbalanced workload distribution.
- Excessive synchronization points.

Memory management

Best practices

- Minimize CPU-GPU transfers
- Batch small transfers together
- Use registers/shared memory for frequently accessed data
- Coalesce global memory accesses when possible
- Balance register usage vs occupancy
- Profile memory access patterns to identify bottlenecks
- Pinned Memory for faster transfers
- Asynchronous transfers with streams to overlap computation

References

NVIDIA developers, 2025. <https://developer.nvidia.com/>

CUDA Toolkit Documentation v12.8 Update 1, 2025. <https://docs.nvidia.com/cuda/>

<https://forms.gle/joyhJefKfNX6tpcX8>

